independent service or into a library that you can import into the project as a dependency. You should carefully evaluate the latter option to prevent the system from becoming a *distributed monolith*.

NOTE Thinking about how your code is organized into codebases and repositories can help you focus more on the system architecture and identify those parts that might actually stand on their own as independent services. If this is done correctly, the codebase's organization can favor modularity and loose coupling.

According to the 15-Factor methodology, each codebase should be mapped to an application, but nothing is said about repositories. You can decide to track each codebase in a separate repository or in the same one. Both options are used in the cloud native business. Throughout the book, you'll build several applications, and I recommend you track each codebase in its own Git repository because it'll improve maintainability and deployability.

In the previous chapter, you initialized the first application in the Polar Bookshop system, Catalog Service, and placed it in a `catalog-service` Git repository. I recommend you use GitHub for storing your repositories because later we're going to use GitHub Actions as the workflow engine to define deployment pipelines in support of continuous delivery.

3.1.2 Dependency management with Gradle and Maven

How you manage dependencies for your applications is relevant because it affects their reliability and portability. In the Java ecosystem, the two most used tools for dependency management are Gradle and Maven. Both provide the functionality to declare dependencies in a manifest and download them from a central repository. The reason for listing all the dependencies your